

Объектно- Ориентированное Программирование

(МИСиС, зима-весна 2026)

Полевой Дмитрий Валерьевич

к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

`std::optional`

```
template<class T> class optional;
```

- опциональное значение типа T
- может быть не определено (`std::nullopt`)

`std::optional` (интерфейс)

- **проверка доступности значения**
 - `operator bool()`
 - `bool has_value()`
- **доступ к значению**
 - `T* operator->()`
 - `T& operator*()`

`std::optional` (интерфейс)

- доступ к значению, или исключение
 - `T& value()`
- получить значение, или значение указанное
 - `T value_or(T)`

чем лучше чем
`std::unique_ptr<T>`
?

`std::optional` (производительность)

- хранение в стеке
- +1 байт (+ выравнивание) $> \text{sizeof}(T)$
- семантика значения (копирование)
- экземпляр создается при обращении
(в отличии от указателей)

`std::optional` (для чего используют)

- писать exception-free код
- возврат значения или признака ошибки (если нет выделенного значения)
- опциональный параметр
- ленивая инициализация

`std::optional`

- требуют аккуратности
 - `std::optional<bool>`
 - `std::optional<T*>`
- повод задуматься «чего вы хотите»

ваши вопросы

Lazy initialization

(отложенная/ленивая инициализация)

- ресурсоёмкая операция выполняется непосредственно перед тем, как будет использован её результат

Lazy initialization

(плюсы)

- ускоряется начальное создание
- «тяжелые вычисления» выполняются «по требованию», а не «всегда»

Lazy initialization

(минусы)

- невозможно явно задать порядок инициализации
- возникает задержка при «первом обращении»

Lazy initialization

(варианты реализации)

- через указатель
- `std::optional`

```
class Service {
    std:: unique_ptr<S> s_;
public:
    void do_work() {
        if (!s_) {
            s_ = std::make_unique<S>();
        }
        s_>run();
    }
};
```

```
class Service {  
    std::optional<S> s_;  
public:  
    void do_work() {  
        if (!s_.has_value()) {  
            s_.emplace();  
        }  
    }  
};
```

что не так с примерами?

Что не так с примерами

- инициализация часто требует параметров
- в примерах не показано
 - как они передаются
 - где они хранятся до момента инициализации

Важные оговорки

- вам чаще придется «вписываться в чужой код»
- писать сразу «правильно» сложно (рефакторинг)
- хотите использовать – ищите готовое и качественное (библиотеки)
- примеры в лекциях - иллюстрации
 - крайне схематичны
 - не учитывают многих моментов

Важные оговорки

- мы рассматриваем
«паттерны внутри приложения»
- некоторые возможны
«внутри системы приложений»
- про более сложные системы читайте
«архитектура корпоративных приложений»

ваши вопросы

Singleton

(Синглтон, Одиночка)

- порождающий шаблон проектирования
- идея
- ровно один экземпляр
- легко доступный всем клиентам
- создаваемый по необходимости (lazy initialization) если получится

Singleton

(примеры использования)

- "узкое горлышко" к общему ресурсу
- логирование
- обработка событий (очередь сообщений)
- менеджер памяти
- black-board прототипирование

Singleton

- может рассматриваться как замена набору функций, разделяющих данные
- существуют реализации, допускающие наследование и/или инстанцирование

Singleton

(технические вопросы)

- порядок создания static объектов
- порядок уничтожения static объектов
- линковка нескольких компонент
- МНОГОПОТОЧНОСТЬ

Singleton

(особенности реализации)

```
S (const S&) = delete;
```

```
S& operator=(const S&) = delete;
```

```
S (S&&) = delete;
```

```
S& operator=(S&&) = delete;
```

```
private:
```

```
S ();
```

```
~S ();
```

Singleton

(реализация Майерса)

```
public:  
    static S& instance() {  
        static S obj;  
        return obj;  
    }
```

Singleton Майерса

- начиная со стандарта C++11 является потокобезопасным и не создает проблемы блокировок
- остается проблема с порядком уничтожения объектов

Singleton

(leaky-вариант)

```
public:  
    static S& instance() {  
        static S* obj = new S();  
        return *obj;  
    }
```

Singleton

(leaky-вариант)

```
public:  
    static S& instance() {  
        static unique_ptr<S> obj;  
        return *obj;  
    }
```

Singleton

(использование)

```
S::instance().go();
```

Singleton

(минусы)

- в идеале состояние синглтона не должно влиять на результат работы программы
- скрывает наличие зависимостей
- усложняет тестирование
- усложняет изменения в архитектуре

Singleton

должен оставаться одиначкой

если синглтонов несколько,
возможно, пора оставить одного

который будет отвечать за порядок
создания и уничтожения остальных

ваши вопросы

Observer

(паттерн)

- изменение состояния — уведомить других
- состав подписчиков меняется в runtime

примеры:

- GUI: кнопка нажата -> обновить другие виджеты
- изменились показания: пересчитать/проверить/обновить

Observer

(терминология)

- много вариантов названий
 - события и подписчики (event, subscriber)
 - вещатель и слушатели (broadcaster, listeners)
 - сигналы и слоты (signal, slot)
- тесно связан с «настоящими ООП языками»
 - не везде есть вызов функций

Observer

(терминология)

- издатель – сообщает о событии
- подписчик – должен получить сообщение
- подписка – включение подписчика в получатели
- отписка – исключение подписчика из получателей

Наивное решение

```
class App {  
    UIPanel*   panel;  
    UILogger*  logger;  
public:  
    void on_app_open();  
};
```

Наивное решение

```
void App::on_app_open() {  
    panel->update();  
    logger->log("app open");  
    // новый подписчик → правим этот метод  
}
```

Observer

(паттерн - проблемы)

- издатель знает обо всех подписчиках напрямую
- сложно
 - читать
 - модифицировать
 - тестировать
 - командно разрабатывать

Observer

(паттерн - идея)

- общий интерфейс подписчика
- издатель хранит список подписчиков
- издатель не знает, кто именно в списке
- список динамический (подписка/отписка)
- надо оповестить – сделать по списку

Варианты реализации

- виртуальный вызов
- функтор (callable object)

Интерфейс (подписчика)

```
struct Msg; // сообщение
```

```
struct IObserver {  
    virtual void notify(const Msg& msg) = 0;  
    virtual ~IObserver() = default;  
};
```

Реализация

```
class UILogger  
    : public IObserver {  
    void notify(const Msg& msg) override;  
}
```

Интерфейс (издателя)

```
class EventSystem {  
    std::vector<IObserver*> observers_;  
public:  
    void subscribe(IObserver* o);  
    void unsubscribe(IObserver* o);  
    void on_app_open(const Msg& msg);  
}
```

Реализация

```
void EventSystem::subscribe (IObserver* o) {  
    observers_.push_back(o);  
}
```

```
void EventSystem::unsubscribe (IObserver* o) {  
    std::erase(observers_, o);  
}
```

Реализация

```
void EventSystem::on_app_open(const Msg& msg) {  
    for (auto* o : observers_)  
        o->notify(msg);  
}  
};
```

Интерфейс (функтор)

```
using struct Msg; // сообщение
```

```
using H = std::function<void(const Msg&)>
```

Реализация

```
class EventSystem {  
    std::vector<H> handlers_;  
public:  
    void subscribe(F h);  
    void on_app_open(const Msg& msg);  
};
```


Реализация

```
void EventSystem::subscribe(F h) {  
    handlers_.push_back(std::move(h));  
}  
  
void EventSystem::on_app_open(const Msg& msg) {  
    for (auto& h : handlers_)  
        h(msg);  
}
```

Использование

```
EventSystem es;
```

```
es.subscribe([] (const Msg msg) {  
    panel.update(msg); } );
```

```
es.subscribe([] (const Msg msg) {  
    logger.log(msg); } );
```

Реализация (функтор)

- нет иерархии и виртуального вызова
- подписка проще – любой callable
- отписка сложнее (и требует доделок примера)
 - нужен уникальный идентификатор подписчика
 - `shared_ptr` на обработчик

Рекомендации

(когда использовать)

- один объект должен уведомлять произвольное число других
- состав подписчиков меняется в runtime
- издатель не должен знать о конкретных подписчиках
- нужна событийная модель framework-ов (GUI, игры, reactive-системы)

Рекомендации

(когда избегать)

- состав подписчиков небольшой и не меняется
- уведомления синхронны
- цепочка событий детерминирована

ваши вопросы

Mediator

(паттерн)

- компоненты должны координироваться, не зная друг о друге напрямую

пример:

UI форма

- поля ввода, кнопки, чекбоксы
- элементы зависят друг от друга (доступность, значения)

Mediator

(решаемая проблема)

- сильная связность
 - все «знают» о всех ($N \rightarrow$ до N^2 зависимостей)
 - системы напрямую вызывают методы друг друга
 - логика координации состояний размазана по всем компонентам
 - новая система = правка ВСЕХ источников событий

Mediator

(идея)

- изолировать логику координации компонент
- компоненты
 - знают о медиаторе
 - информируют медиатор
- медиатор координирует

Mediator

(когда использовать)

- выделение упрощает (логика в одном месте)
- компонент много, зависимости взаимные
- логика координации переписывается без исправления компонент

Mediator

(когда избегать)

- выделение усложняет
- логика координации тривиальна
- компонент мало (2-3) и не будет роста числа
- компоненты независимы

Mediator

(проблемы)

- God Object
 - разбиение по функциональным группам
- зацикливание (рекурсия) уведомлений
 - очередь для уведомлений
 - явный режим координации

Event Bus

(паттерн)

- несколько независимых подсистем
- подсистемы не должны напрямую «знать» друг о друге
- системы в общем «информационном поле»
- события должны приводить к изменениям

Event Bus

(решаемая проблема)

- **сильная СВЯЗНОСТЬ**
 - все «знают» о всех
 - системы напрямую вызывают методы друг друга
 - новая система = правка ВСЕХ источников событий
- **порядок уведомлений жестко в коде**

Event Bus

(паттерн - идея)

- Observer – частичное решение
 - явная регистрация каждого подписчика у каждого издателя
- Event Bus – выделенный хранитель связей
 - ведение реестра издателей
 - ведение реестра подписчиков
 - пересылка сообщений

Event Bus

(варианты реализации)

- виртуальный вызов
 - адреса объектов для идентификации
- функтор (callable object)
 - дополнительные механизмы идентификации

Event Bus

(проблемы)

- «погибшие» подписчики
 - отписка в деструкторе подписчика
- зацикливание (рекурсия) – публикация события в процессе рассылки
- God Object
 - разделение одной шины на несколько (домены)

ваши вопросы

Command

(паттерн)

- хранение «действия» в экземпляре

пример:

- асинхронность (разделение извещения и обработки)
- история операций (undo/redo)

Command

(решаемая проблема)

- логика изменений «размазана»
- повторить операцию программно — нет механизма

Command

(идея)

- изменение = последовательность действий
- действие
 - экземпляр (параметры в полях)
 - парные методы «применить»/«отменить»
 - можно хранить, передавать, применять

Command

(варианты реализации)

- виртуальный вызов
 - интерфейс (execute/undo)
- функтор (callable object)
 - с состоянием и/или захватом контекста

Command

(проблемы)

- сложный контекст применения команды
 - хранение в команде «снимка» состояния
- время жизни участников
 - `shared_ptr`, `weak_ptr`
- неограниченный рост истории
 - ручное ограничения

Command

(когда использовать)

- нужна
 - история операций (undo/redo)
 - транзакционность
 - повтор последовательности операций
 - логгирование
 - очередь/отложенность обработки

Command

(когда избегать)

- выделение усложняет
- операция выполняется единожды
- повторы не требуются
- один источник вызова
- достаточно прямого вызова

ваши вопросы